CS108 Project Report: Mini Game Hub

Ritwik Khandelwal
25B1020

Shreyas Lohiya
25B1094

CSE Department
Indian Institute Of Technology Bombay

Contents

1 Introduction **2**

2 External Libraries **3**

3 Design & Architecture **3**
3.1 Key Architectural Decisions 4

4 Features Implemented **4**
4.1 Authentication System (Bash) 4
4.2 Main Menu & Theme Selection 4
4.3 Resizable Pygame window 5
4.4 Chain Reaction 5
4.5 In-Game Controls 5
4.6 Leaderboard 5
4.7 Statistics Plots 5
4.8 Animation as time-based interpolation 5

5 Directory Structure **5**

6 Installation & Setup **6**

7 Usage **6**

8 Retrospective **7**
8.1 Challenges 7
8.2 What We Would Do Differently 8

9 References **8**

Introduction

Mini Game Hub is a two-player, turn-based game collection that runs on Linux. The project combines a Bash-based terminal front-end for authentication and navigation with a Python/Pygame graphical back-end for all gameplay. Two players log in (or sign up) through the terminal, choose a game and a visual theme, and then compete in a resizable Pygame window. Results are persisted to a CSV history file, which feeds a colourful leaderboard and a set of Matplotlib analytics plots.

The hub currently hosts four classic strategy games:

- **Tic-Tac-Toe** – 10×10 board, five-in-a-row variant.
- **Othello** – Standard 8×8 reversi rules.
- **Connect4** – 7×7-column drop-token game with physics animation.
- **Chain Reaction** – Explosive chain-reaction strategy on a 9×6 grid.

All games share a common object-oriented architecture through an abstract `Game` base class, ensuring consistency and ease of extension.

External Libraries

Library	Version	Purpose
pygame-ce	$\geq$ 2.5.7	The primary framework for GUI rendering, event handling, and managing the game loop
numpy	$\geq$ 2.2.6	Board representation, vectorised win-checking in games and animation math
matplotlib	$\geq$ 3.10.8	In-game plots for top players across games, most played games by frequency, win distribution, PvP heat map
csv	stdlib	Reading `history.csv` inside the `plotstuff` method to aggregate win/loss/draw data without an external database.
datetime	stdlib	Stamping each game record with `date.today()` before appending to `history.csv`.
subprocess	stdlib	Launching `leaderboard.sh` from within Python, and removing the temporary `plot.png` after it has been loaded into Pygame.
sys	stdlib	Reading command-line arguments (`sys.argv`) to receive player usernames from the Bash launcher.
sha256sum	—	Password encryption by converting plaintext into sha256 encoded string
column	—	Terminal table formatting

Design & Architecture

Terminal UI – Interactive Terminal UI for signup and login with ASCII art and ANSI coloring

Theme selection – Four background themes (Phonk, Futuristic, Retro, Space) selectable before each game session, including an option to select if you want the board to shake

Animated game elements –

- Connect4: physics-accurate falling token (gravity + bounce damping).
- Tic-Tac-Toe / Connect4: animated win-line that draws itself progressively.
- Othello: ellipse-based coin-flip animation when pieces are outflanked.
- Chain Reaction: timed explosion animation with balls travelling to neighbours.

In-game controls – Resign button (forfeit immediately), Claim Draw button with Accept/Reject dialogue, and a Quit confirmation overlay.

Animated UI elements – Menu title pulses sinusoidally; buttons scale up smoothly on hover with eased interpolation.

Key Architectural Decisions

Inheritance-based game abstraction All four games extend `game_template`. The base class owns the main pygame loop, common UI elements (resign, claim draw, quit overlay), board panning via WASD, and board shaking. Each subclass only implements `win_check`, `draw_check`, `valid_check`, `move`, `drawboard`, `specificmousepressevents`, and 2 or 3 helper functions for game logic and specific animations. This makes adding a new game straightforward and keeps the event loop DRY.

Responsive layout Every size — cell dimensions, font size, button positions — is derived from a single `smallerscalevalue` computed from the current window size relative to the reference resolution of 1280×720. A `VIDEORESIZE` event triggers `update()` which recomputes all layout values, making the application fully resizable.

CSV history log Game results are appended to `history.csv` in a simple five-field format: winner, loser, date, game name, is-draw flag. This flat file is parsed by both Python (for matplotlib plots) and awk (for the leaderboard), avoiding any database dependency.

Single Game loop This starts in the base class and calls the child classes and separate draw functions in the main loop. This maintains modularity of the code and truncates the loading time for repeated pygame initilizations.

A button class This class enables easy addition of buttons anywhere in the code which saves time for adding changes and new implementations.

Features Implemented

Authentication System (Bash)

New users can sign up with a username (4–12 alphanumeric/underscore characters, must start with a letter) and a password (8–20 characters, must include upper, lower, digit, and one of `_@#$%^&*`). Passwords are stored as SHA-256 hashes in `users.tsv`. Login validates the hash. The second player cannot reuse the first player's username. A custom `readpass` function masks input with bullet characters.

Main Menu & Theme Selection

After authentication the Python engine presents a game-selection menu followed by a theme-selection menu. Themes correspond to a background image and an audio track loaded at runtime, allowing new themes to be added very easily by dropping files into the `images/` and `audio/` directories and adding a string in line of code as

```
theme=menu(self.screen,"CHOOSE A THEME","Phonk",
"Futuristic","Retro","Space","new-theme-name")
```

Resizable Pygame window

: All board positions, cell sizes, fonts, and button geometry scale with the window to make it look playable and resize the side according to your preference at the same time.

Chain Reaction

Chain-Reaction is implemented in addition to the three specified games, it is a cult classic phone game which has an interesting recursive logic and a decent animation implementation.

In-Game Controls

All games share resign buttons (immediately awards win to the opponent), claim-draw buttons (opponent must accept or reject), a quit-confirmation overlay, and WASD / arrow-key board panning.

Leaderboard

Accessible both from the shell (before launching Python) and from the post-game menu. Metrics: Username, Wins, Win%, Losses, Loss%, Wins/Losses. Data is broken down per game and combined. The terminal table is rendered with box-drawing characters and ANSI colour codes using awk.

Statistics Plots

Four matplotlib views: Top 5 Players (bar chart), Total Wins (pie), Most Played Games (pie), and a PvP heat map showing the net win differential between every pair of players. Plots are saved to a temporary PNG and displayed inside the pygame window.

Animation as time-based interpolation

Animations (falling tokens in Connect4, flipping discs in Othello, exploding orbs in Chain Reaction, win-line drawing in Tic-Tac-Toe / Connect4) are driven by elapsed wall-clock time rather than frame counters, so they behave consistently regardless of frame rate.

Directory Structure

```
mini-game-hub/
|-- main.sh              # Shell entry point
|-- leaderboard.sh       # Terminal leaderboard renderer
|-- game.py              # Python entry point: GameEngine, stat plots
|-- history.csv          # Append-only game result log
|-- users.tsv            # Username and SHA-256 password hash store
|-- games/
```

```
|   |-- game_template.py # Abstract base class; owns the main pygame loop
|   |-- tictactoe.py     # 10x10 five-in-a-row Tic-Tac-Toe
|   |-- othello.py       # 8x8 Othello
|   |-- connect4.py      # 7x7 Connect4 with physics drop animation
|   `-- chainreaction.py # 9x6 Chain Reaction with cascade explosions
|-- ui/
|   |-- button.py        # Hover-animated Button widget
|   |-- menu.py          # Reusable option-menu screen
|   |-- welcome.py       # Welcome splash screen
|   `-- thankyou.py      # Exit / thank-you screen
|-- images/              # Background PNGs per theme; X/O/board token images
|-- audio/               # .ogg background music tracks (one per theme)
`-- fonts/               # BlackgothRegular.otf display font
```

Installation & Setup

Requirements: Python 3.10+, Bash, standard POSIX utilities (`awk`, `sort`, `column`, `sha256sum`). Install dependencies from requirements.txt:

```
pip install -r requirements.txt
```

```
pygame-ce==2.5.7
numpy==2.2.6
matplotlib==3.10.8
```

Make shell scripts executable and run:

```
chmod +x main.sh leaderboard.sh
./main.sh
```

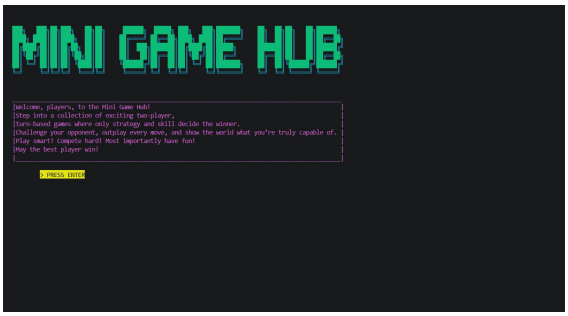
Assets: The `images/`, `audio/`, and `fonts/` directories must be present alongside the scripts.

Usage

Run `./main.sh` from the project root in a terminal. The shell script walks through the following flow:

2. **Game selection** — choose Tic-Tac-Toe, Othello, Connect4, or Chain Reaction.
3. **Theme selection** — choose Phonk, Futuristic, Retro, or Space.
4. **Gameplay** — click cells to place pieces. Use WASD / arrow keys to pan the board. Resign or claim draw via the side buttons. Press Escape for the quit overlay.
5. **Post-game** — view the leaderboard, view plots, play again, or quit.

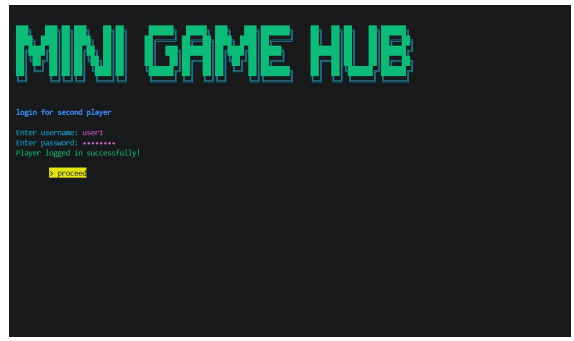
Keyboard shortcuts: W/A/S/D or arrow keys pan the board. Escape opens the quit-confirmation overlay.



(a) Welcome screen — press Enter to continue.

(b) Play or Leaderboard — view leaderboard or proceed to play

(c) Authentication — players sign up



(d) Players log in

Figure 1: Application flow screens

Retrospective

Challenges

Resizable Window Scaling – The most tedious challenge was making every UI element board cells scale correctly when the user resizes the Pygame window. A single `smallerscalevalue` factor was adopted, and all pixel measurements were multiplied by it one by one which was a tedious task. The `update()` method on `Game` reconstructs every button and reloads the font at the new size upon each `VIDEORESIZE` event.

Othello Turn Skipping and Game End – Standard Othello requires passing a turn when a player has no legal moves, and ending the game when *both* players are locked. The overridden `switch_turn()` method handles this by probing all cells for valid moves after each switch, and calling `win_check()` only when neither player can act.

Modular Implementation of Animation – Implementing Animation as a loop in a separate function outside the game loop proved to be useless as it stopped running the original loop and freezed the board. Adding the code directly in the game loop was also ugly and depreived the code of its modularity. So a function made for drawing each frame separately in the loop on the basis of when the bool value which changed on need to do basis and worked smoothly.

Single Game Loop – Required effective directory management and proper function and class definitions across co-classes and child-classes

Physics animation in Connect4 – Simulating realistic bouncing without a physics engine required deriving closed-form expressions for the position of the token at each frame using kinematics equations, a coefficient of restitution, and a bounce-count cap to prevent infinite micro-bounces.

Team-Work

What We Would Do Differently

1. **Replay system.** Record every move to the history file and build a replay viewer that steps through past games frame-by-frame, which would be especially compelling for Chain Reaction chains.
2. **Undo/take-back mechanic.** Maintain a move stack per game and allow both players to agree to undo the last move – useful for casual play. Also the move stack could be used for reviewing game like in chess for analysis later.

References

1. Pygame documentation — https://www.pygame.org/docs/, https://pyga.me/
2. NumPy documentation — https://numpy.org/doc/ and Bodhitree
3. Matplotlib documentation — https://matplotlib.org/stable/ and Bodhitree
4. *Chain Reaction* game rules — https://en.wikipedia.org/wiki/Chain_Reaction_(game)
5. *Reversi / Othello* game rules — https://en.wikipedia.org/wiki/Reversi
6. ANSI escape codes reference — https://en.wikipedia.org/wiki/ANSI_escape_code
7. GNU awk manual — https://www.gnu.org/software/gawk/manual/ and Bodhitree
8. Background Images - AI Generated